M05W04 - Optimizer

Pham Bao Long

November 23, 2024

1 Introduction

In this assignment, I aim to present and analyze fundamental optimization algorithms commonly used in deep learning. The complete implementation of the project is available on my GitHub repository.

2 Implementation

2.1 Gradient Descent

Optimization function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

Gradient Descent Function:

$$W_t = W_{t-1} - \alpha \cdot \nabla f(W_{t-1})$$

Where:

- W_{t-1}: The previous weight or parameter being optimized.
- α: The learning rate, controlling the step size.
- $\nabla f(W)$: The gradient of the optimize function with respect to W.

Implementation:

1. **Step 1: Compute the derivative of the optimization function.**
   The optimization function is given by:

   $$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

   The partial derivatives with respect to w_1 and w_2 are:

   $$\frac{\partial f(w_1, w_2)}{\partial w_1} = 0.2w_1$$

   $$\frac{\partial f(w_1, w_2)}{\partial w_2} = 4w_2$$

   Combine into the Gradient Vector:

   $$\nabla f(W_{t-1}) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \end{bmatrix} = \begin{bmatrix} 0.2w_1 \\ 4w_2 \end{bmatrix}$$

   However, the input is an array:

   $$W_{t-1} = [w_1, w_2]$$

   Then, the gradient is computed as:

   $$\nabla f(W_{t-1}) = \begin{bmatrix} w_1 & w_2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

   For example,

   $$W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$$

   $$\implies \nabla f(W_{t-1}) = \begin{bmatrix} -5 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -1 & -8 \end{bmatrix}$$

2. **Step 2: Use the Gradient Descent function to update the value of W.**
Continuing the previous example, we apply the Gradient Descent function to the optimization problem.

Given:

- $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$ (The previous weight or parameter being optimized)
- $\alpha = 0.01$ (The learning rate, controlling the step size)
- $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$ (The gradient of the optimization function with respect to W_{t-1})

The updated W_i is computed as:

$$W_t = W_{t-1} - \alpha \cdot \nabla f(W)$$

Substituting the values, we get:

$$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.01 \cdot \begin{bmatrix} -1 & -8 \end{bmatrix} = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix}$$

3. **Step 3: Repeat the process using the updated W.**
Continuing with the previous example, the updated W_t is now redefined as W_{t-1} for the next iteration.
Given:

- $W_{t-1} = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix}$ (The previous weight or parameter being optimized)
- $\alpha = 0.01$ (The learning rate, controlling the step size)

The gradient now is computed:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} -4.99 & -1.92 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -0.998 & -7.68 \end{bmatrix}$$

The updated value of W_i is:

$$W_t = W_{t-1} - \alpha \cdot \nabla f(W_{t-1})$$

$$W_t = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix} - 0.01 \cdot \begin{bmatrix} -0.998 & -7.68 \end{bmatrix} = \begin{bmatrix} -4.98002 & -1.8432 \end{bmatrix}$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.
Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```python
import numpy as np

def df_W(W):
    """
    Calculate the gradient of the function f(w1, w2) = 0.1w1^2 + 2w2^2.
    """
    # Coefficient matrix for the diagonal terms in the gradient
    coefficient = np.diag([0.2, 4])
    # Gradient calculation
    dw = np.dot(W, coefficient)
    return dw

def sgd(W, dW, lr):
    """
    Update weights using Stochastic Gradient Descent (SGD).
    """
    W = W - lr * dW
    return W
```

Listing 1: Python Code for Gradient Descent

```python
    def train_pl(optimizer, lr, epochs):
    """
    Train the model over a given number of epochs,
    using the provided optimizer and learning rate.
    """
    # Initial weights
    W = np.array([-5, -2], dtype=np.float32)
    results = [W]  # List to store W over all epochs

    for epoch in range(epochs):
        # Compute the gradient
        dW = df_W(W)
        # Update weights using the optimizer
        W = optimizer(W, dW, lr)
        # Store the updated weights
        results.append(W)

    return results

# Training process
weights_history = train_pl(sgd, lr=0.01, epochs=30)

# Print the results
for epoch, W in enumerate(weights_history):
    print(f"Epoch {epoch}: W = {W}")
```

Listing 2: Python Code for Gradient Descent

2.2 Momentum-based Gradient Descent

Optimization function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

$$\implies \nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Momentum update rule:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1})$$

Momentum-based Gradient Descent function:

$$W_t = W_{t-1} - \alpha \cdot V_t$$

Implementation:

1. **Step 1: Compute the derivative of the optimization function.**
   Given:

   - $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$ (The previous weight or parameter being optimized)

   The gradient is computed as:

   $$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

   Substituting W_{t-1}:

   $$\implies \nabla f(W_{t-1}) = \begin{bmatrix} -5 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -1 & -8 \end{bmatrix}$$

2. **Step 2: Compute V_t (Velocity update):**
   Given:

   - β: The momentum rate, which determines the influence of the previous velocity on the current update.
   - V_{t-1}: The initial velocity, set to zero for all dimensions.
   - $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to W_{t-1}.

   The velocity is computed using the momentum update rule as follows:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

   where $\beta = 0.9$, $V_{t-1} = \begin{bmatrix} 0 & 0 \end{bmatrix}$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$. Substituting these values:

$$V_t = 0.9 \cdot \begin{bmatrix} 0 & 0 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -1 & -8 \end{bmatrix}$$

$$\implies V_t = \begin{bmatrix} 0 & 0 \end{bmatrix} + 0.1 \cdot \begin{bmatrix} -1 & -8 \end{bmatrix} = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}.$$

3. **Step 3: Update Weights (Momentum-Based Gradient Descent):**
   Given:

   - W_{t-1}: The initial weights.
   - α: The learning rate, which controls the step size.
   - V_t: The updated velocity.

   The weights are updated using the momentum-based gradient descent formula:

$$W_t = W_{t-1} - \alpha \cdot V_t,$$

   where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$, $\alpha = 0.1$, and $V_t = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}$. Substituting these values:

$$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.1 \cdot \begin{bmatrix} -0.1 & -0.8 \end{bmatrix} = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix}$$

4. **Step 4: Repeat the Process with Updated W:**
   Continuing from the first update, W_t and V_t is now redefined as W_{t-1} and V_{t-1} respectively for the next iteration.
   Given:

   - W_{t-1}: The previous updated weight vector.
   - V_{t-1}: The velocity from the previous iteration.
   - α: The consistent learning rate used for this iteration.
   - β: The consistent momentum coefficient.

   First, compute the gradient:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix},$$

   where $W_{t-1} = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix}$. Substituting values:

$$\nabla f(W_{t-1}) = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -0.998 & -7.68 \end{bmatrix}.$$

Next, compute the velocity:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

where $\beta = 0.9$, $V_{t-1} = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -0.998 & -7.68 \end{bmatrix}$. Substituting these values:

$$V_t = 0.9 \cdot \begin{bmatrix} -0.1 & -0.8 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -0.998 & -7.68 \end{bmatrix}$$

$$\implies V_t = \begin{bmatrix} -0.09 & -0.72 \end{bmatrix} + \begin{bmatrix} -0.0998 & -0.768 \end{bmatrix} = \begin{bmatrix} -0.1898 & -1.488 \end{bmatrix}.$$

Finally, update the weights:

$$W_t = W_{t-1} - \alpha \cdot V_t,$$

where $W_{t-1} = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix}$, $\alpha = 0.1$, and $V_t = \begin{bmatrix} -0.1898 & -1.488 \end{bmatrix}$. Substituting these values:

$$W_t = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix} - 0.1 \cdot \begin{bmatrix} -0.1898 & -1.488 \end{bmatrix}$$

$$\implies W_t = \begin{bmatrix} -4.99 & -1.92 \end{bmatrix} + \begin{bmatrix} 0.01898 & 0.1488 \end{bmatrix} = \begin{bmatrix} -4.97102 & -1.7712 \end{bmatrix}.$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.
Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```python
import numpy as np

def df_W(W):
    """
    Compute the gradient of the function f with respect to W.
    """
    return np.dot(W, np.diag([0.2, 4]))

def velocity(dW, V, momentum):
    """
    Update the velocity using the momentum term.
    """
    return momentum * V + (1 - momentum) * dW

def sgd_momentum(W, lr, Vel):
    """
    Update weights using momentum-based gradient descent.
    """
    return W - lr * Vel
```

Listing 3: Python Code for Momentum-based Gradient Descent

```python
def train_pl(lr, momentum, epochs):
    """
    Train a model using momentum-based stochastic gradient descent.

    Parameters:
        lr (float): Learning rate.
        momentum (float): Momentum coefficient.
        epochs (int): Number of epochs to train.

    Returns:
        list: A list of weights at each epoch.
    """
    # Initialize weights and velocity
    W = np.array([-5, -2], dtype=float)
    V = np.zeros_like(W)
    results = [W.copy()]

    for epoch in range(epochs):
        # Compute the gradient
        dW = df_W(W)

        # Update the velocity
        V = velocity(dW, V, momentum)

        # Update weights
        W = sgd_momentum(W=W, lr=lr, Vel=V)
        results.append(W.copy())

        # Print progress
        print(f"Epoch {epoch + 1}:")
        print(f"  Gradient (dW): {np.round(dW, 5)}")
        print(f"  Velocity (V): {np.round(V, 5)}")
        print(f"  Updated Weights (W): {np.round(W, 5)}\n")

    return results

# Run the training loop
train_pl(lr=0.1, momentum=0.9, epochs=30)
```

Listing 4: Python Code for Momentum-based Gradient Descent

2.3 Root Mean Square Propagation (RMSProp)

Optimization function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$

$$\implies \nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Exponential Moving Average function (EMA):

$$S_t = \gamma \cdot S_{t-1} + (1-\gamma) \cdot (\nabla f(W_{t-1}))^2,$$

RMSProp optimizer:

$$W_t = W_{t-1} - \alpha \cdot \frac{\nabla f(W_{t-1})}{\sqrt{S_t + \epsilon}}$$

Implementation:

1. **Step 1: Compute the gradient (Derivative of optimization function):**
   Given:

   - W_{t-1}: The initial weights being optimized.

   The gradient is computed as:

   $$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

   where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$:

   $$\implies \nabla f(W_{t-1}) = \begin{bmatrix} -5 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -1 & -8 \end{bmatrix}$$

   .

2. **Step 2: Calculate the Exponential Moving Average (EMA):**
   Given:

   - S_{t-1}: The initial value of the EMA, set to neutral.
   - γ: The decay rate, determining the weight of past values.
   - $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to the initial weights.

   The EMA is computed as:

   $$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

   where $S_{t-1} = \begin{bmatrix} 0 & 0 \end{bmatrix}$, $\gamma = 0.9$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$. Substituting these values:

   $$S_t = 0.9 \cdot \begin{bmatrix} 0 & 0 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -1 & -8 \end{bmatrix}^2$$

   $$\implies S_t = \begin{bmatrix} 0 & 0 \end{bmatrix} + \begin{bmatrix} 0.1 & 6.4 \end{bmatrix} = \begin{bmatrix} 0.1 & 6.4 \end{bmatrix}.$$

3. **Step 3: Update the weights using the RMSProp optimizer:**
   Given:

   - W_{t-1}: The initial weights.
   - $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to W_{t-1}.
   - α: The learning rate of the optimizer.
   - S_t: The updated exponential moving average of squared gradients.
   - ϵ: A small positive constant to prevent division by zero.

   The updated weights are computed as:

   $$W_t = W_{t-1} - \alpha \cdot \frac{\nabla f(W_{t-1})}{\sqrt{S_t + \epsilon}}$$

   where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$, $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$, $\alpha = 0.1$, $S_t = \begin{bmatrix} 0.1 & 6.4 \end{bmatrix}$, and $\epsilon = 0.001$. Substituting these values:

   $$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.1 \cdot \frac{\begin{bmatrix} -1 & -8 \end{bmatrix}}{\sqrt{\begin{bmatrix} 0.1 & 6.4 \end{bmatrix} + 0.001}}$$

   $$\implies W_t = \begin{bmatrix} -4.68534 & -1.6838 \end{bmatrix}.$$

4. **Step 4: Repeat the Process with the Updated Weights:**
At this step, the newly updated W_t and S_t are redefined as W_{t-1} and S_{t-1}, respectively, to prepare for the next iteration.

Given:

- W_{t-1}: The previous weight vector after the last update.
- S_{t-1}: The exponential moving average (EMA) from the previous iteration.
- α: The consistent learning rate for the optimizer.
- γ: The decay rate of the EMA.
- ϵ: A small positive constant to prevent division by zero.

First, compute the gradient:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix},$$

where $W_{t-1} = \begin{bmatrix} -4.68534 & -1.6838 \end{bmatrix}$. Substituting the values:

$$\nabla f(W_{t-1}) = \begin{bmatrix} -4.68534 & -1.6838 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -0.93707 & -6.73519 \end{bmatrix}.$$

Next, compute the updated EMA:

$$S_t = \gamma \cdot S_{t-1} + (1-\gamma) \cdot (\nabla f(W_{t-1}))^2,$$

where $S_{t-1} = \begin{bmatrix} 0.1 & 6.4 \end{bmatrix}$, $\gamma = 0.9$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -0.93707 & -6.73519 \end{bmatrix}$. Substituting these values:

$$S_t = 0.9 \cdot \begin{bmatrix} 0.1 & 6.4 \end{bmatrix} + (1-0.9) \cdot \begin{bmatrix} -0.93707 & -6.73519 \end{bmatrix}^2 = \begin{bmatrix} 0.17781 & 10.29628 \end{bmatrix}.$$

Finally, compute the updated weight:

$$W_t = W_{t-1} - \alpha \cdot \frac{\nabla f(W_{t-1})}{\sqrt{S_t + \epsilon}},$$

where $W_{t-1} = \begin{bmatrix} -4.68534 & -1.6838 \end{bmatrix}$, $\nabla f(W_{t-1}) = \begin{bmatrix} -0.93707 & -6.73519 \end{bmatrix}$, $\alpha = 0.1$, $S_t = \begin{bmatrix} 0.17781 & 10.29628 \end{bmatrix}$, and $\epsilon = 0.001$. Substituting the values:

$$W_t = \begin{bmatrix} -4.68534 & -1.6838 \end{bmatrix} - 0.1 \cdot \frac{\begin{bmatrix} -0.93707 & -6.73519 \end{bmatrix}}{\sqrt{\begin{bmatrix} 0.17781 & 10.29628 \end{bmatrix} + 0.001}} = \begin{bmatrix} -4.46374 & -1.47391 \end{bmatrix}.$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.
Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```python
import numpy as np

def compute_gradient(W):
    """
    Compute the gradient of the objective function with respect to W.
    Here, we assume a simple linear function with a diagonal matrix for simplicity.
    """
    return np.dot(W, np.diag([0.2, 4]))
```

Listing 5: Python Code for Root Mean Square Propagation (RMSProp)

```python
def update_ema(S, decay_rate, gradient_squared):
    """
    Update the Exponential Moving Average (EMA) of squared gradients.

    Parameters:
        S (ndarray): The previous EMA of squared gradients.
        decay_rate (float): The decay factor for EMA.
        gradient_squared (ndarray): The square of the gradients.

    Returns:
        ndarray: The updated EMA of squared gradients.
    """
    return decay_rate * S + (1 - decay_rate) * gradient_squared

def rmsprop_update(W, lr, gradient, ema, epsilon=0.001):
    """
    Perform a single update using the RMSProp optimizer.

    Parameters:
        W (ndarray): The current weights.
        lr (float): The learning rate.
        gradient (ndarray): The gradient of the objective function.
        ema (ndarray): The Exponential Moving Average of squared gradients.
        epsilon (float): A small constant to prevent division by zero.

    Returns:
        ndarray: The updated weights.
    """
    return W - lr * (gradient / np.sqrt(ema + epsilon))

def train_rmsprop(lr, decay_rate, epochs):
    """
    Train a simple model using the RMSProp optimization algorithm.

    Parameters:
        lr (float): The learning rate for RMSProp.
        decay_rate (float): The decay rate for EMA.
        epochs (int): The number of training iterations (epochs).
    """
    # Initialize weights (W) and EMA (S) for RMSProp
    W = np.array([-5, -2])
    S = np.zeros_like(W)  # EMA initialized to zero
    results = [W.copy()]

    # Training loop
    for epoch in range(epochs):
        # Compute gradient
        gradient = compute_gradient(W)

        # Update EMA with squared gradients
        S = update_ema(S, decay_rate, gradient**2)

        # Update weights using RMSProp
        W = rmsprop_update(W, lr, gradient, S)

        # Save the updated weights for tracking
        results.append(W.copy())

        # Print progress (optional for monitoring training)
        print(f"Epoch {epoch + 1}:")
        print(f"  Gradient (dW): {np.round(gradient, 5)}")
        print(f"  EMA (S): {np.round(S, 5)}")
        print(f"  Updated Weights (W): {np.round(W, 5)}\n")

    return results

# Call the training function with parameters
train_rmsprop(lr=0.1, decay_rate=0.9, epochs=30)
```

Listing 6: Python Code for Root Mean Square Propagation (RMSProp)

2.4 Adaptive Moment Estimation (Adam)

Optimization Function:

$$f(w_1, w_2) = 0.1w_1^2 + 2w_2^2$$
$$\implies \nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

Momentum Update Rule:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1})$$

Exponential Moving Average (EMA):

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

Bias Correction for Velocity (Momentum Update Rule):

$$V_t^{\text{corr}} = \frac{V_t}{1 - \beta^t}$$

Bias Correction for Moving Average (EMA):

$$S_t^{\text{corr}} = \frac{S_t}{1 - \gamma^t}$$

Adaptive Moment Estimation Update rule:

$$W_t = W_{t-1} - \alpha \cdot \frac{V_t^{\text{corr}}}{\sqrt{S_t^{\text{corr}}} + \epsilon}$$

Implementation:

1. **Step 1: Compute the gradient (Derivative of optimization function):**
   Given:

   - W_{t-1}: The initial weights being optimized.

   The gradient is computed as:

   $$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

   where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$:

   $$\implies \nabla f(W_{t-1}) = \begin{bmatrix} -5 & -2 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -1 & -8 \end{bmatrix}$$

   .

2. **Step 2: Compute V_t (Velocity update):**
   Given:

   - β: The momentum rate, which determines the influence of the previous velocity on the current update.
   - V_{t-1}: The initial velocity, set to zero for all dimensions.
   - $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to W_{t-1}.

   The velocity is computed using the momentum update rule as follows:

   $$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

   where $\beta = 0.9$, $V_{t-1} = \begin{bmatrix} 0 & 0 \end{bmatrix}$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$. Substituting these values:

   $$V_t = 0.9 \cdot \begin{bmatrix} 0 & 0 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -1 & -8 \end{bmatrix}$$

   $$\implies V_t = \begin{bmatrix} 0 & 0 \end{bmatrix} + 0.1 \cdot \begin{bmatrix} -1 & -8 \end{bmatrix} = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}.$$

3. **Step 3: Calculate the Exponential Moving Average (EMA):**
Given:

- S_{t-1}: The initial value of the EMA, set to neutral.
- γ: The decay rate, determining the weight of past values.
- $\nabla f(W_{t-1})$: The gradient of the optimization function with respect to the initial weights.

The EMA is computed as:

$$S_t = \gamma \cdot S_{t-1} + (1-\gamma) \cdot (\nabla f(W_{t-1}))^2$$

where $S_{t-1} = \begin{bmatrix} 0 & 0 \end{bmatrix}$, $\gamma = 0.999$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -1 & -8 \end{bmatrix}$. Substituting these values:

$$S_t = 0.999 \cdot \begin{bmatrix} 0 & 0 \end{bmatrix} + (1-0.999) \cdot \begin{bmatrix} -1 & -8 \end{bmatrix}^2$$

$$\implies S_t = \begin{bmatrix} 0 & 0 \end{bmatrix} + \begin{bmatrix} 0.001 & 0.064 \end{bmatrix} = \begin{bmatrix} 0.001 & 0.064 \end{bmatrix}.$$

4. **Step 4: Calculate Bias Correction of Velocity and EMA:**

a. Bias-Correction of Velocity:
Given:

- V_t: The updated velocity used for correction.
- β: The momentum coefficient used for this correction.
- t: The index of the iteration.

The bias correction of velocity is computed as:

$$V_t^{\text{corr}} = \frac{V_t}{1-\beta^t}$$

where $V_t = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}$, $\beta = 0.9$, and $t = 1$. Substituting these values:

$$V_t^{\text{corr}} = \frac{\begin{bmatrix} -0.1 & -0.8 \end{bmatrix}}{1-0.9^1} = \begin{bmatrix} -1.0 & -8.0 \end{bmatrix}$$

b. Bias-Correction of Moving Average:
Given:

- S_t: The updated moving average used for this correction.
- γ: The decay rate used for this correction.
- t: The index of the iteration.

The bias correction of the moving average is computed as:

$$S_t^{\text{corr}} = \frac{S_t}{1-\gamma^t}$$

where $S_t = \begin{bmatrix} 0.001 & 0.064 \end{bmatrix}$, $\gamma = 0.999$, and $t = 1$. Substituting these values:

$$S_t^{\text{corr}} = \frac{\begin{bmatrix} 0.001 & 0.064 \end{bmatrix}}{1-0.999^1} = \begin{bmatrix} 1.0 & 64.0 \end{bmatrix}.$$

5. **Step 5: Update the weights using the Adam optimizer: Given:**

   - W_{t-1} The initial weights.
   - V_t^{corr}: The Bias correction of the Velocity.
   - S_t^{corr}: The Bias correction of the Moving Average.
   - α: The consistent learning rate.
   - ϵ: The small positive constant

   The weights is updated as:

$$W_t = W_{t-1} - \alpha \cdot \frac{V_t^{\text{corr}}}{\sqrt{S_t^{\text{corr}}} + \epsilon}$$

   where $W_{t-1} = \begin{bmatrix} -5 & -2 \end{bmatrix}$, $V_t^{\text{corr}} = \begin{bmatrix} -1.0 & -8.0 \end{bmatrix}$, $S_t^{\text{corr}} = \begin{bmatrix} 1.0 & 64.0 \end{bmatrix}$, $\alpha = 0.1$, and $\epsilon = 0.001$. Substituting those values:

$$W_t = \begin{bmatrix} -5 & -2 \end{bmatrix} - 0.1 \cdot \frac{\begin{bmatrix} -1.0 & -8.0 \end{bmatrix}}{\sqrt{\begin{bmatrix} 1.0 & 64.0 \end{bmatrix}} + 0.001} = \begin{bmatrix} -4.9001 & -1.90001 \end{bmatrix}$$

6. **Step 6: Repeat the Process with the Updated Weights:**
   For the second iteration, W_t, V_t, and S_t are redefined as W_{t-1}, V_{t-1}, and S_{t-1}. Especially, t is set to be 2. **Given:**

   - W_{t-1}: The previous weight vector after the last update.
   - $Vt-1$: The velocity from the previous iteration.
   - β: The consistent momentum coefficient.
   - S_{t-1}: The exponential moving average (EMA) from the previous iteration.
   - γ: The decay rate of the EMA.
   - α: The consistent learning rate for the optimizer.
   - ϵ: A small positive constant to prevent division by zero.

   Firstly, compute the gradient:

$$\nabla f(W_{t-1}) = W_{t-1} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix}$$

   where $W_{t-1} = \begin{bmatrix} -4.9001 & -1.90001 \end{bmatrix}$:

$$\implies \nabla f(W_{t-1}) = \begin{bmatrix} -4.9001 & -1.90001 \end{bmatrix} \cdot \begin{bmatrix} 0.2 & 0 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} -0.98002 & -7.60005 \end{bmatrix}.$$

   Secondly, compute Velocity and Moving Average:

   Velocity is computed as:

$$V_t = \beta \cdot V_{t-1} + (1 - \beta) \cdot \nabla f(W_{t-1}),$$

   where $\beta = 0.9$, $V_{t-1} = \begin{bmatrix} -0.1 & -0.8 \end{bmatrix}$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -0.98002 & -7.60005 \end{bmatrix}$. Substituting these values:

$$V_t = 0.9 \cdot \begin{bmatrix} -0.1 & -0.8 \end{bmatrix} + (1 - 0.9) \cdot \begin{bmatrix} -0.98002 & -7.60005 \end{bmatrix} = \begin{bmatrix} -0.188 & -1.48 \end{bmatrix}$$

   Moving Average is computed as:

$$S_t = \gamma \cdot S_{t-1} + (1 - \gamma) \cdot (\nabla f(W_{t-1}))^2$$

   where $S_{t-1} = \begin{bmatrix} 0.001 & 0.064 \end{bmatrix}$, $\gamma = 0.999$, and $\nabla f(W_{t-1}) = \begin{bmatrix} -0.98002 & -7.60005 \end{bmatrix}$. Substituting these values:

$$S_t = 0.999 \cdot \begin{bmatrix} 0.001 & 0.064 \end{bmatrix} + (1 - 0.999) \cdot \begin{bmatrix} -0.98002 & -7.60005 \end{bmatrix}^2 = \begin{bmatrix} 0.00196 & 0.1217 \end{bmatrix}$$

Thirdly, compute bias correction of velocity and moving average:

Bias correction of velocity:

$$V_t^{\text{corr}} = \frac{V_t}{1-\beta^t}$$

where $V_t = \begin{bmatrix}-0.188 & -1.48\end{bmatrix}$, $\beta = 0.9$, and $t = 1$. Substituting these values:

$$V_t^{\text{corr}} = \frac{\begin{bmatrix}-0.188 & -1.48\end{bmatrix}}{1-0.9^2} = \begin{bmatrix}-0.98948 & -7.7895\end{bmatrix}$$

Bias correction of Moving Average:

$$S_t^{\text{corr}} = \frac{S_t}{1-\gamma^t}$$

where $S_t = \begin{bmatrix}0.00196 & 0.1217\end{bmatrix}$, $\gamma = 0.999$, and $t = 1$. Substituting these values:

$$S_t^{\text{corr}} = \frac{\begin{bmatrix}0.00196 & 0.1217\end{bmatrix}}{1-0.999^2} = \begin{bmatrix}0.98021 & 60.87882\end{bmatrix}.$$

Finally, update new weights:

$$W_t = W_{t-1} - \alpha \cdot \frac{V_t^{\text{corr}}}{\sqrt{S_t^{\text{corr}}} + \epsilon}$$

where $W_{t-1} = \begin{bmatrix}-4.9001 & -1.90001\end{bmatrix}$, $V_t^{\text{corr}} = \begin{bmatrix}-0.98948 & -7.7895\end{bmatrix}$, $S_t^{\text{corr}} = \begin{bmatrix}0.98021 & 60.87882\end{bmatrix}$, $\alpha = 0.1$, and $\epsilon = 0.001$. Substituting those values:

$$W_t = \begin{bmatrix}-4.9001 & -1.90001\end{bmatrix} - 0.1 \cdot \frac{\begin{bmatrix}-0.98948 & -7.7895\end{bmatrix}}{\sqrt{\begin{bmatrix}0.98021 & 60.87882\end{bmatrix}} + 0.001} = \begin{bmatrix}-4.80026 & -1.80019\end{bmatrix}$$

Notation: This process can be repeated iteratively until the optimal W_i is obtained.
Here is the reference Python code for the above calculation. It implements the process over 30 epochs and returns a list of W values for every update:

```python
import numpy as np

def df_W(W):
    """Compute the gradient of W."""
    return np.dot(W, np.diag([0.2, 4]))

def velocity(V, momentum, dW):
    """Update the velocity with momentum."""
    V = momentum * V + (1 - momentum) * dW
    return V

def EMA(S, decay_rate, dW):
    """Compute the Exponential Moving Average (EMA) of the gradient squares."""
    S = decay_rate * S + (1 - decay_rate) * (dW ** 2)
    return S

def V_corr(V, momentum, epoch):
    """Apply bias correction to the velocity."""
    V_corr = V / (1 - momentum ** epoch)
    return V_corr

def S_corr(S, decay_rate, epoch):
    """Apply bias correction to the EMA."""
    S_corr = S / (1 - decay_rate ** epoch)
    return S_corr
```

Listing 7: Python Code for Apdative Moment Estimate Optimizer (Adam)

```python
def Adam(W, lr, V_corr, S_corr, epsilon=0.001):
    """Update weights using the Adam optimizer."""
    W = W - lr * (V_corr / (np.sqrt(S_corr) + epsilon))
    return W

def train_pl(lr, momentum, decay_rate, epochs):
    """Train the model with Adam optimizer."""
    # Initial weights, EMA, and velocity
    W = np.array([-5, -2])
    S = np.zeros_like(W)
    V = np.zeros_like(W)
    result = [W]

    # Training loop
    for epoch in range(1, epochs + 1):
        # Compute gradient
        dW = df_W(W)

        # Update velocity (momentum)
        V = velocity(V, momentum, dW)

        # Update EMA of squared gradients
        S = EMA(S, decay_rate, dW)

        # Apply bias correction
        v_corr = V_corr(V, momentum, epoch)
        s_corr = S_corr(S, decay_rate, epoch)

        # Update weights using Adam optimizer
        W = Adam(W, lr, v_corr, s_corr)

        # Store the updated weights
        result.append(W.copy())

        # Print progress for monitoring
        print(f"Epoch {epoch}:")
        print(f"  Gradient (dW): {np.round(dW, 5)}")
        print(f"  Velocity (V): {np.round(V, 5)}")
        print(f"  EMA (S): {np.round(S, 5)}")
        print(f"  Velocity_corr (V_corr): {np.round(v_corr, 5)}")
        print(f"  EMA_corr (S_corr): {np.round(s_corr, 5)}")
        print(f"  Updated Weights (W): {np.round(W, 5)}\n")

    return result

# Run the training for 2 epochs with the specified parameters
train_pl(lr=0.1, momentum=0.9, decay_rate=0.999, epochs=30)
```

Listing 8: Python Code for Apdative Moment Estimate Optimizer (Adam)